

WikiBridge

零代码内网穿透本地知识库平台

软件设计规格说明书

项目名称 零代码内网穿透本地知识库平台 WikiBridge
文档类型 软件设计规格说明书 (SDS)
开发单位 武汉大学软件工程课程设计项目组
文档版本 V1.0

文档变更历史记录

序号	变更日期	变更人员	变更内容详情描述	版本
1	2026/06/14	全员	撰写《WikiBridge 软件设计规格说明书》初稿	V1.0

目录

文档变更历史记录	I
1 引言	1
1.1 编写目的	1
1.2 读者对象	1
1.3 软件项目概述	1
1.4 文档概述	1
1.5 定义	2
1.6 参考资料	2
2 软件设计约束	2
2.1 软件设计目标和原则	2
2.2 软件设计的约束和限制	3
3 软件设计	3
3.1 软件体系结构设计	3
3.2 用户界面设计	5
3.3 用例设计	7
3.4 核心流程设计	10
3.5 类设计	10
3.6 数据设计	13
3.6.1 本地数据结构	13
3.6.2 服务器控制面数据表	13
3.6.3 永久数据的操作	14
3.6.4 数据安全设计	14
3.7 部署设计	15
3.8 异常处理与可靠性设计	15
3.9 测试设计	16
结论	16

1、引言

1.1 编写目的

软件设计过程包括体系结构设计、用户界面设计、用例设计、子系统/构件设计、类设计和数据设计，分别从不同层次（从宏观到微观、从全局到局部）、不同视角（从结构到行为、从模块到数据）对软件系统进行设计，产生不同的软件制品（如体系结构模型、用例实现模型、用户界面模型、数据设计模型、部署模型等）。在完成上述设计工作之后，需要将这些设计成果整合为一个系统、完整的设计方案。本文档以软件设计规格说明书的形式描述 **WikiBridge** 平台的设计方案，方便评审人员对设计方案的正确性、合理性等方面进行评审，并为后续编码实现提供蓝图。

1.2 读者对象

用户、软件设计人员、程序员、软件需求分析人员、质量保证人员、软件测试工程师、配置管理工程师，以及关注 WikiBridge 平台架构与实现的课程指导教师。

1.3 软件项目概述

- **项目名称：**零代码内网穿透本地知识库平台 WikiBridge
- **用户单位：**教师、文字工作者、研究者及中小团队等非技术背景的知识工作者
- **开发单位：**武汉大学软件工程课程设计项目组
- **软件项目的背景和大致功能：**

在当前数字化学习与办公场景中，个人和小团队会持续积累大量资料——文档、网页剪藏、聊天记录、课堂笔记、灵感片段等，但它们分散在不同软件与文件夹中，难以长期沉淀，也难以快速找到知识之间的联系。项目组观察到三类核心痛点：**知识零散难归档**、**AI 知识库部署门槛高**、**本地知识库共享困难**。现有 RAG 或私有知识库方案通常需要配置 Python 环境、向量数据库、模型服务甚至 GPU，对非技术用户并不友好；即使在本地搭建了知识库，也很难安全、低成本地发布到公网供他人访问。

WikiBridge 的目标不是单纯“做一个知识库”，而是让普通用户能够**零代码**完成从资料整理、知识结构化到公网访问的完整闭环。平台将知识库生成过程类比为“软件编译”：用户把原始资料放入本地素材目录，主动触发 **Compile**，系统调用大模型 Agent 对素材进行阅读、提炼、去重与结构化，输出带有双向链接（`[[wiki-links]]`）的 Markdown Wiki；本地 Wiki 经轻量 Web 服务渲染为静态站点，再借助 **frp** 隧道将本地站点映射到公网；外部访问者通过浏览器浏览知识图谱，并可使用 Chatbox 进行问答。其核心优势在于：用户数据与主要计算过程留在本地，云端只负责账号、鉴权、端口和流量调度，不保存用户知识内容，从而兼顾低成本、隐私性与易用性。

1.4 文档概述

本文档主要包含两部分内容：

1. **软件设计约束部分：**包括软件设计目标和原则、软件设计受到的约束和限制。

2. **软件设计部分**：主要分为软件体系结构设计、用户界面设计、用例设计、类设计、数据设计以及部署设计。

1.5 定义

术语	含义
Add	素材箱管理阶段，用户将原始资料放入本地 <code>/raw</code> 目录。
Compile	本地知识编译阶段，调用 LLM Agent 将 <code>/raw</code> 素材重构为结构化 Markdown Wiki，写入 <code>/wiki</code> 。
Lint	图谱检查与维护阶段，扫描 <code>/wiki</code> ，修复死链接、孤立页面与冗余结构。
Publish	无感公网发布阶段，将本地静态站经 frp 隧道映射为可访问的公网 URL。
LLM Agent	大语言模型“知识主编”，负责素材阅读、实体提取、去重合并与页面生成。
wiki-links	形如 <code>[[概念]]</code> 的双向链接，用于在 Wiki 页面之间建立可跳转的知识关联。
frp / frpc / frps	开源内网穿透工具 frp；frpc 为客户端，运行在用户端；frps 为服务端，运行在云服务器。
控制面 / 数据面	控制面指账号、鉴权、端口与隧道调度；数据面指用户的知识内容。WikiBridge 将二者隔离。

1.6 参考资料

1. 软件工程. 齐治昌, 谭庆平, 宁洪. 北京: 高等教育出版社, 2012.
2. 需求分析与设计. 马素霞译. 北京: 机械工业出版社, 2009.
3. 面向使用的软件设计. 刘正捷译. 北京: 机械工业出版社, 2011.
4. fatedier. frp ——A fast reverse proxy. <https://github.com/fatedier/frp>.
5. WikiBridge 项目简介文档 (doc.md), 武汉大学软件工程课程设计项目组, 2026.

2、软件设计约束

2.1 软件设计目标和原则

软件设计的目标是，根据软件系统的需求（包括功能性需求和非功能性需求），综合考虑软件开发过程中的各种制约因素（如技术、资源、进度等），遵循软件工程的设计原则（如模块化、信息隐藏、问题分解等），给出软件系统的实现解决方案和蓝图，产生可指导编码实现的设计模型及文档。具体而言，WikiBridge 的设计遵循以下策略和原则：

1. 抽象和逐步求精的原则；
2. 模块化与高内聚度、低耦合度的原则——**知识编译模块与 frp 隧道管理模块保持解耦**；
3. 信息隐藏的原则；
4. 多视点以及关注点分离的原则——**控制面与数据面分离**；
5. 软件重用的原则——复用开源 frp、Markdown 渲染器与 LLM 接口；
6. 迭代设计的原则；
7. 可追踪性的原则——设计制品可追溯到需求用例。

2.2 软件设计的约束和限制

- **运行环境要求**：用户端为桌面图形界面应用（Windows / macOS / Linux）；服务器端运行于 Linux 云主机；知识消费端为现代浏览器，无需安装任何软件。
- **开发语言**：网络隧道改造基于 Go (frp)；知识编译器与本地服务基于 Python / Node.js；用户端 GUI 与消费端 Web 前端基于跨平台桌面框架与 HTML/CSS/JavaScript。
- **标准规范**：遵循 RESTful API 设计规范、Markdown / CommonMark 文本规范、OpenAI 兼容的对话接口规范。
- **外部依赖**：开源 frp 隧道、可配置的大语言模型服务（API）、Markdown 静态站渲染器。
- **隐私约束**：云端服务器**不得存储**用户笔记、文档、Wiki 页面或向量数据，用户业务数据主要保留在本地。

3、 软件设计

3.1 软件体系结构设计

WikiBridge 系统整体分为**服务器、用户端、用户的用户（知识消费端）**三个端。服务器承担“控制面”职责——用户认证、Token 鉴权、动态端口/子域名分配、frps 隧道管理与流量统计，但**不保存**用户知识内容；用户端是知识拥有者使用的核心软件，运行知识编译逻辑、本地 Web 服务与网络隧道客户端，对用户屏蔽技术细节；知识消费端无需安装软件，通过浏览器访问公网 URL，浏览静态 Wiki 并使用 Chatbox 问答。系统总体架构见图 3.1.1。

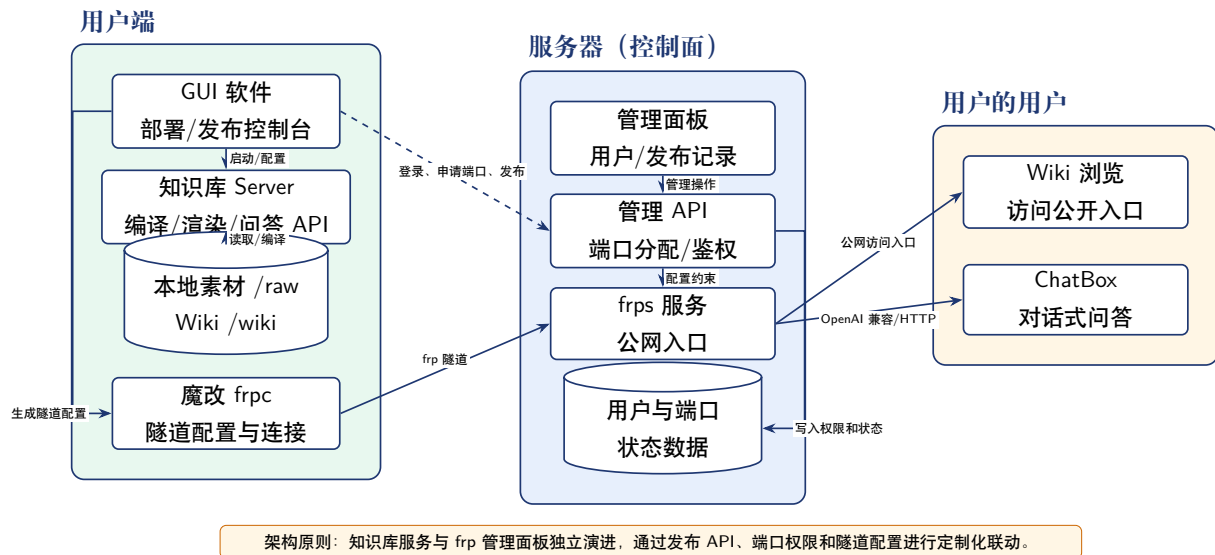


图 3.1.1 WikiBridge 系统的总体体系结构图

(1) 用户端子系统 用户端采用经典三层结构（见图 3.1.2）。界面层提供素材箱、编译控制台、Lint 报告与发布面板；业务逻辑层由 CompileManager、LintManager、PublishManager 等控制类驱动 Add/Compile/Lint/Publish 工作流；基础服务层封装 LLM 客户端、本地 Web Server 与魔改 frpc。该子系统依赖服务器的管理 API 获取发布入口。

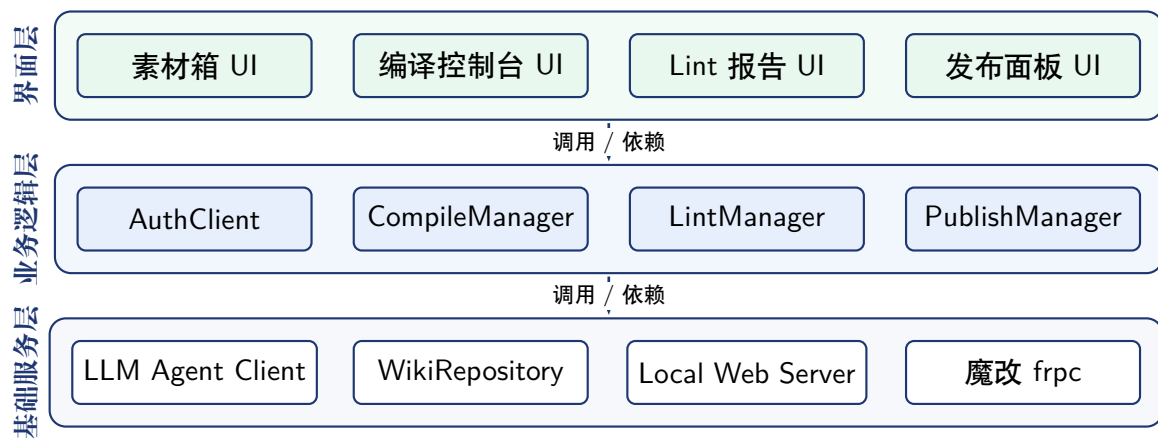


图 3.1.2 “用户端”子系统的分层设计架构

(2) 服务器子系统 服务器只承担控制面职责（见图 3.1.3）。AuthController 负责注册/登录/Token 校验；TunnelController 与 FrpsManager 负责动态分配子域名或端口、维护 frps 与 frpc 的连接状态；TrafficMonitor 统计带宽、连接时长并实施限速；所有状态写入控制面数据库（用户、Token、隧道、发布记录），**与用户知识内容物理隔离**。

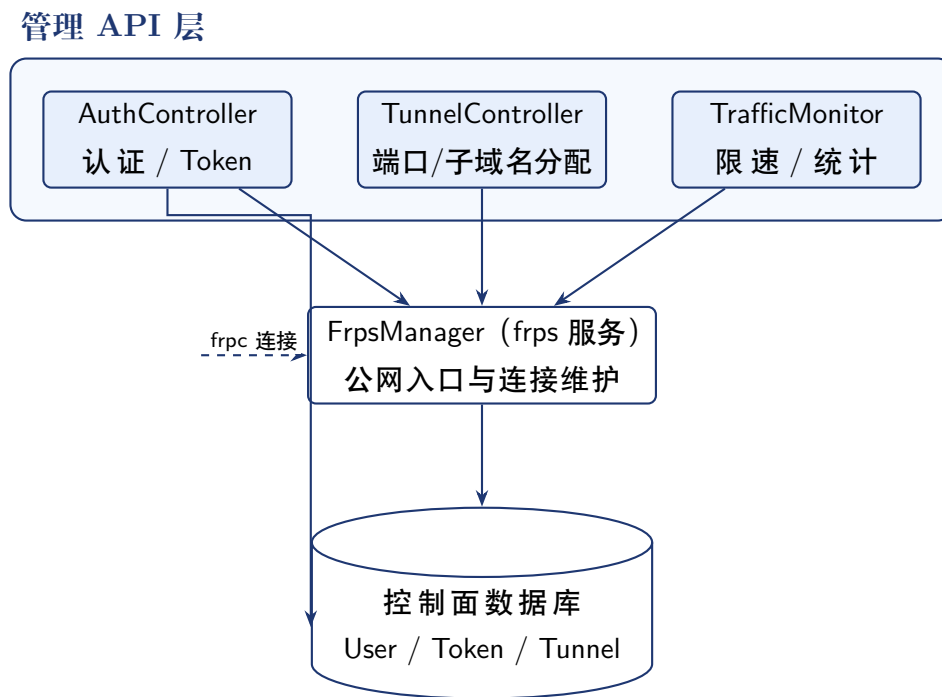


图 3.1.3 “服务器（控制面）”子系统的设计架构

3.2 用户界面设计

根据 WikiBridge 的用例描述及交互流程，用户端 GUI 需要以下一组界面：

- 欢迎/引导界面 GuidingUI：介绍系统用途与“数据本地优先”的隐私边界，引导用户进入工作台。
- 登录界面 LoginUI：输入平台账号并登录，登录后自动申请公网映射。
- 素材箱界面 MaterialBoxUI：管理 /raw 目录，支持拖拽导入未处理资料。
- 编译控制台界面 CompileConsoleUI：触发 Compile，展示实时进度、日志与错误。
- Lint 报告界面 LintReportUI：触发“检查优化”，展示死链接、孤立页面与冗余结构。
- 发布面板界面 PublishPanelUI：展示本地服务状态、隧道状态与公网访问 URL。

知识消费端则包含 WikiBrowseUI（静态图谱浏览与双向链接跳转）与 ChatboxUI（上下文问答与引用展示）。各界面之间的跳转关系见图 3.2.1。

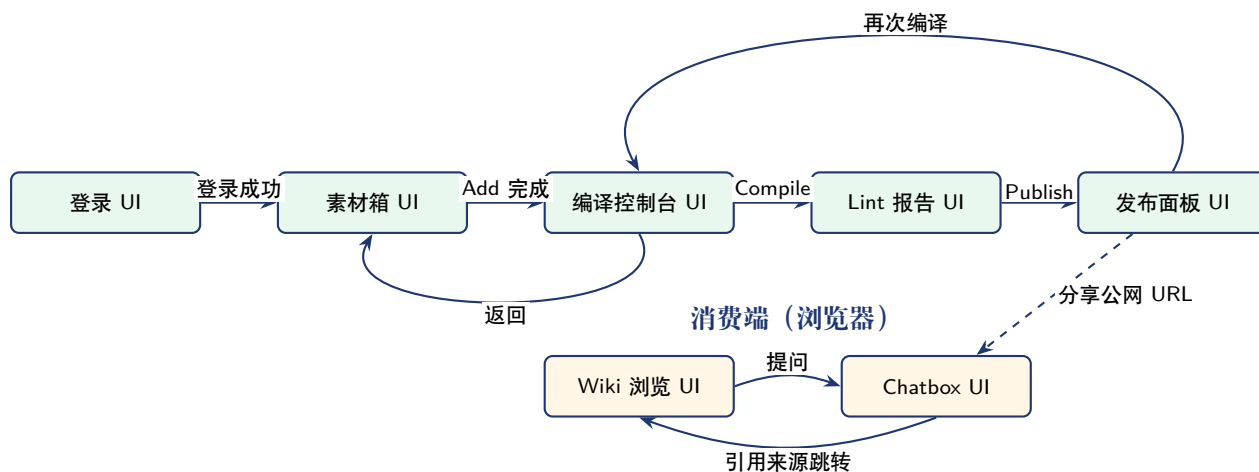


图 3.2.1 用户界面跳转关系图

下面给出若干关键界面的线框设计。图 3.2.2 为“素材箱 + 编译控制台”界面，左侧管理 /raw 素材列表，右侧展示编译进度与实时日志。

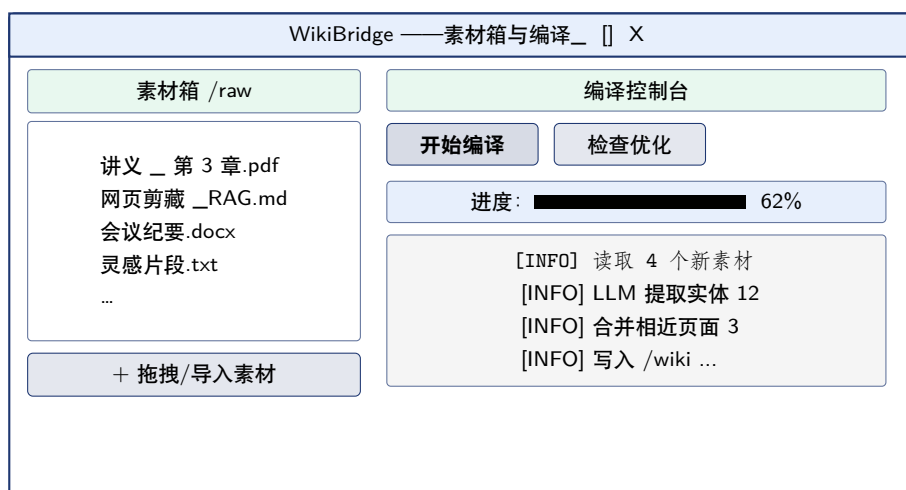


图 3.2.2 “素材箱与编译控制台”界面线框

图 3.2.3 为发布面板，展示本地服务、隧道状态与公网 URL；图 3.2.4 为消费端 Wiki 浏览与 Chatbox 界面。



图 3.2.3 “发布面板” 界面线框

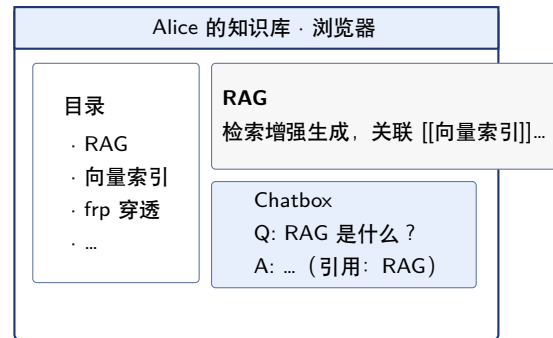


图 3.2.4 消费端“Wiki 浏览 + Chatbox” 界面线框

3.3 用例设计

WikiBridge 的用例总览见图 3.3.1，刻画了三类角色（知识拥有者、平台管理员、知识消费者）与系统边界的关系。本节针对核心用例给出实现级的顺序图与文字说明。

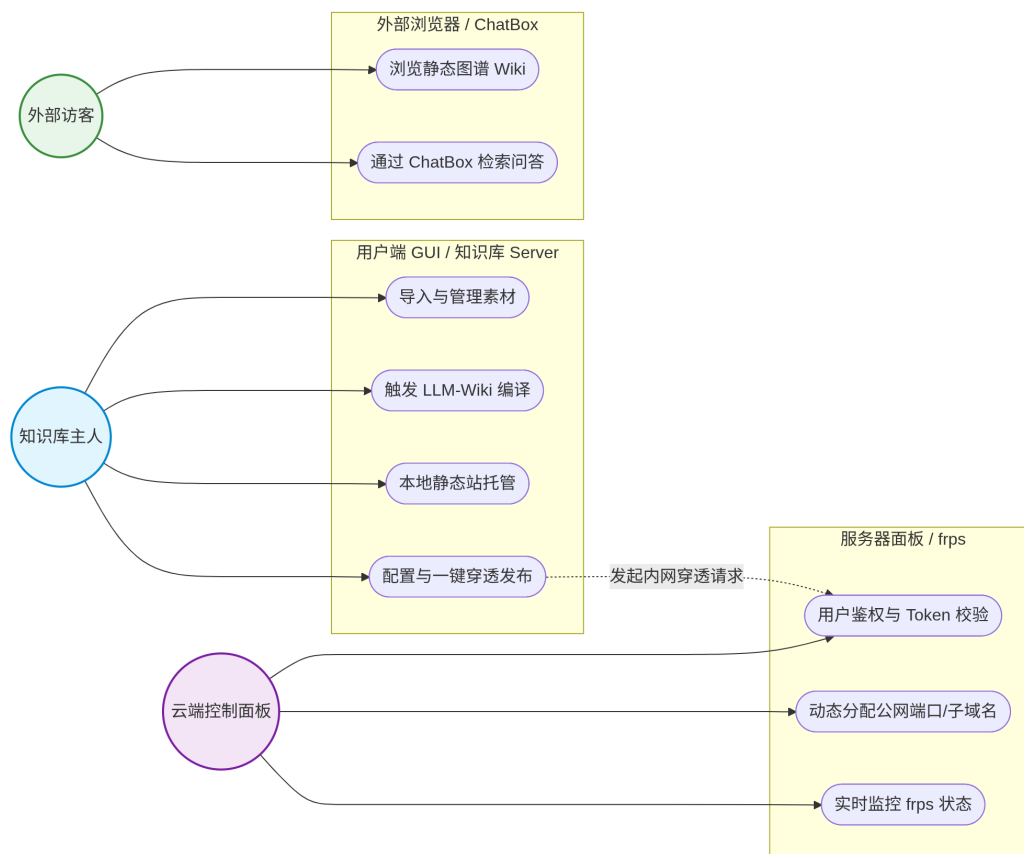


图 3.3.1 WikiBridge 用例图：角色与系统边界

图 3.3.2 给出端到端的发布与访问总流程（素材输入 → 本地编译 → frp 发布 → 浏览器消费）。

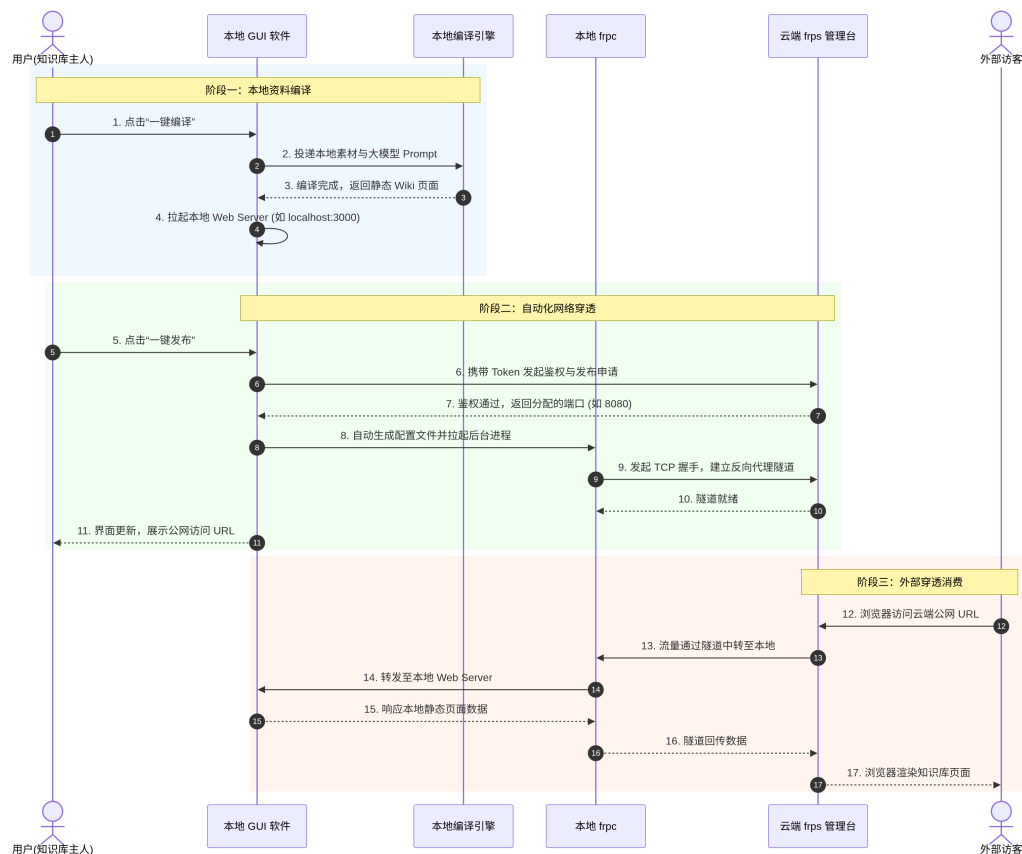


图 3.3.2 端到端发布与访问流程顺序图

(1) “Compile 本地编译”用例实现的设计方案 用户点击“开始编译”后，CompileConsoleUI 向控制类 CompileManager 发送 startCompile(); CompileManager 先扫描 /raw 中的新素材，随后调用 LLMAgentClient 的 extractKnowledge(rawDocs) 完成阅读、实体提取与去重，得到结构化结果后交由 WikiRepository 的 writePages() 写入或合并 /wiki 中的 Markdown 页面并建立 [[wiki-links]]，最终把进度与日志反馈给界面。具体过程见图 3.3.3。

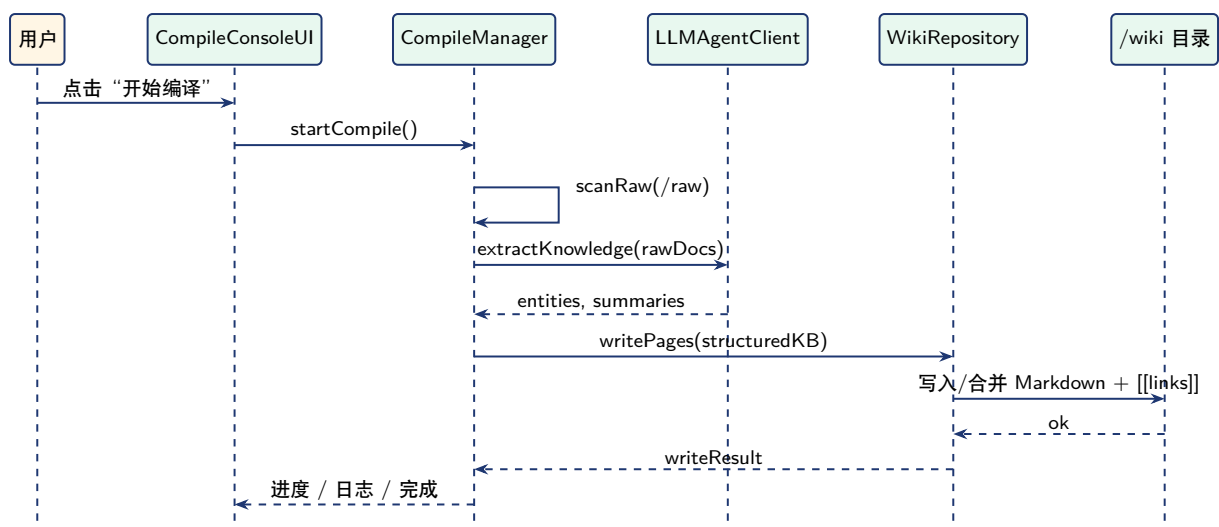


图 3.3.3 “Compile 本地编译”用例实现的顺序图

(2) “Publish 公网发布”用例实现的设计方案 用户登录后点击“发布”，PublishPanelUI 经 AuthClient 向服务器 AuthController 提交账号密码并取得 Token；随后 PublishManager 启动本地 LocalWebServer 渲染 /wiki，并驱动魔改 frpc 携带 Token 向 TunnelController 申请隧道；服务器分配子域名/端口并配置 FrpsManager，frpc 与 frps 建立隧道后回传公网 URL，界面最终向用户展示该 URL。具体过程见图 3.3.4。

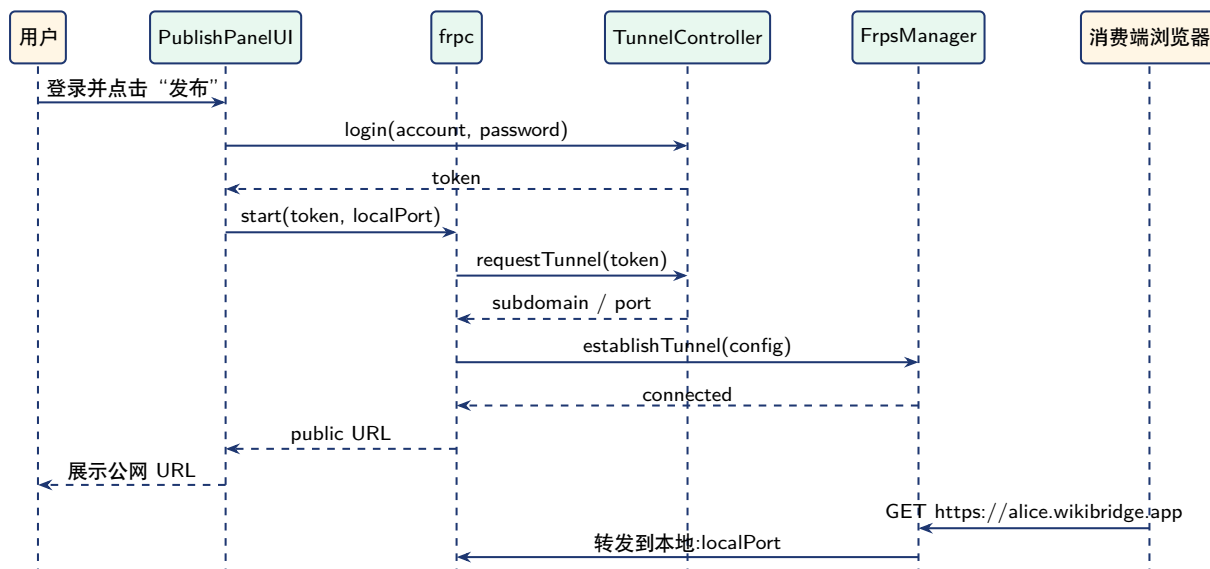


图 3.3.4 “Publish 公网发布”用例实现的顺序图

图 3.3.5 给出“用户端”子系统的设计类图：在分析类的基础上，依据用例实现的交互图增加了 LLMAgentClient、WikiRepository、LocalWebServer、FrpcController 等设计类，以实现知识编译与公网发布。

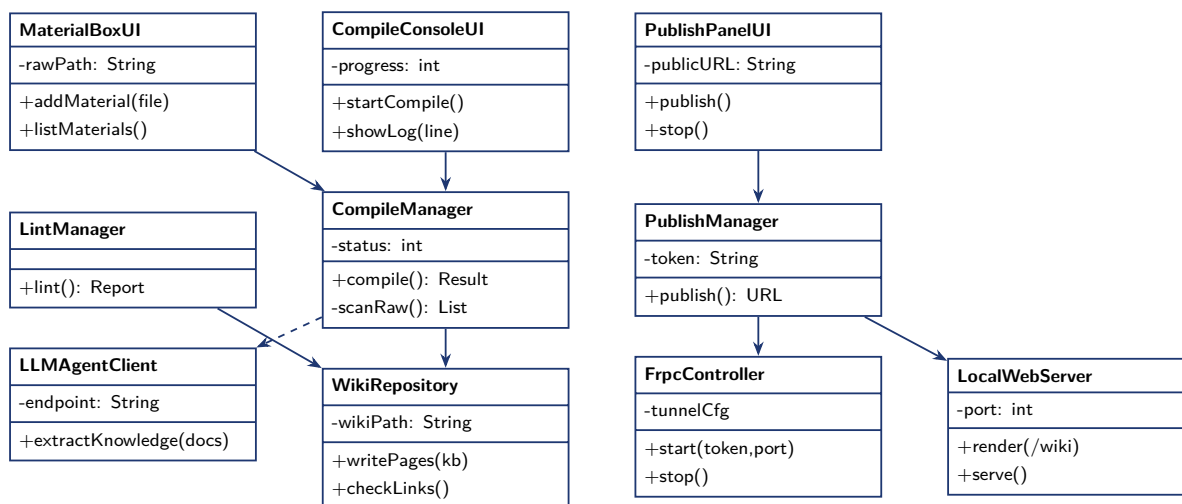


图 3.3.5 “用户端”子系统的设计类图

图 3.3.6 给出“服务器（控制面）”子系统的设计类图。AuthController、TunnelController 等控制类与 UserRepository、TunnelRepository 实体类协作，完成认证、端口分配与状态持久化。

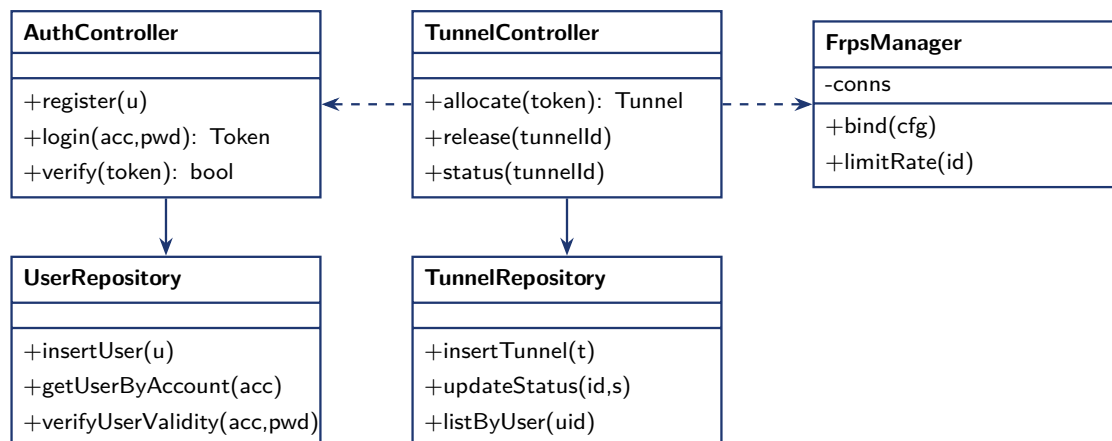


图 3.3.6 “服务器（控制面）”子系统的设计类图

3.4 核心流程设计

WikiBridge 的主流程由用户主动触发的 **Add / Compile / Lint / Publish** 四个阶段串联而成，如图 3.4.1 所示。用户先把原始资料放入本地 **/raw** (Add)，点击编译后由 LLM-Wiki 编译器读取新素材、提炼去重并写入 **/wiki** (Compile)；Lint 检查器扫描 **/wiki** 修复死链接与冗余并**写回**；本地 Web Server 将 **/wiki** 渲染为静态站点，用户确认发布后经 frp 隧道映射为公网 URL (Publish)，外部访问者即可在浏览器中浏览 Wiki 并使用 Chatbox 问答。该流程清晰地区分了 **本地数据面** (Add/Compile/Lint/本地渲染) 与**公网发布** (Publish)，普通浏览不触发任何 Token 消耗。

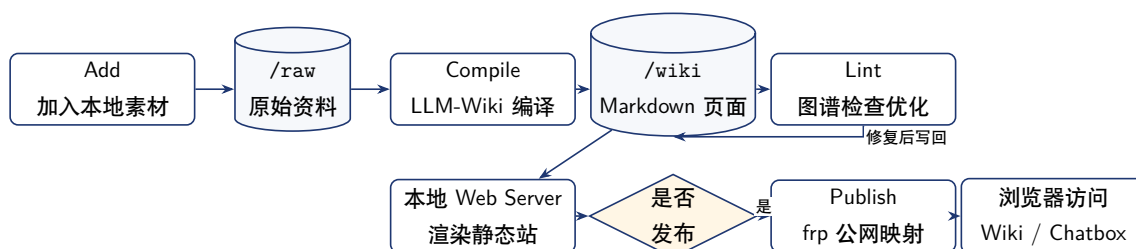


图 3.4.1 Add / Compile / Lint / Publish 核心流程图

3.5 类设计

(1) **精化关键设计类间的关系** 在分析类图中，User、Token、TunnelConfig、WikiPage 等类在设计阶段仍然有意义，成为关键设计类。其关系精化见图 3.5.1：一个 User 可持有多个 Token (一对多)，一个 User 可创建多个 TunnelConfig (一对多)；WikiPage 通过 `[[wiki-links]]` 与其它 WikiPage 形成自关联 (多对多)，由 WikiRepository 统一管理。

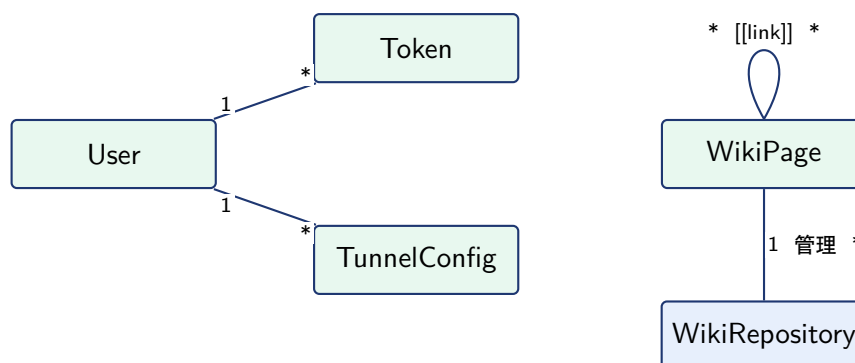


图 3.5.1 精化关键设计类间的关系

(2) **精化关键类属性的设计** 关键设计类的属性精化见表 3.5.1。所有用户私有信息（如密码、Token）可见范围均为 `private`，对外仅通过受控方法访问。

表 3.5.1 关键设计类的属性精化

类	属性	类型 / 可见性	说明
User	account	String / private	用户账号，初值为空串
	password	String / private	用户密码，初值为空串
Token	value	String / private	鉴权令牌，frpc 凭此申请隧道
	expireAt	long / private	过期时间戳
TunnelConfig	subdomain	String / private	分配的子域名
	localPort	int / private	本地服务端口
	status	int / private	隧道状态： IDLE/CONNECTED/CLOSED
WikiPage	title	String / private	页面标题，唯一
	links	List<String> / private	指向其它页面的 <code>[[wiki-links]]</code>

(3) **精化关键类方法的设计** 关键控制类与实体类的方法签名精化见表 3.5.2。

表 3.5.2 关键类方法的精化设计

类	可见性	方法签名与功能
CompileManager	public	Result compile() ——扫描 /raw、调用 LLM、写入 /wiki
	private	List<Doc> scanRaw(String path) ——读取新增素材
LintManager	public	LintReport lint() ——检查死链接、孤立页面与冗余
PublishManager	public	String publish() ——启动本地服务并申请隧道，返回 URL
LLMAgentClient	public	KB extractKnowledge(List<Doc> docs) ——提炼结构化知识
WikiRepository	public	void writePages(KB kb) / bool checkLinks()
AuthController	public	Token login(String acc, String pwd) / bool verify(Token t)
TunnelController	public	Tunnel allocate(Token t) / void release(String id)
UserRepository	public	bool verifyUserValidity(String acc, String pwd)

(4) 构造类的状态图 TunnelConfig（发布隧道）对象在其生命周期中需针对外部事件（登录、断网、回收等）变迁状态，因此为其构造状态图。图 3.5.2 描述了发布隧道从空闲、连接、在线服务到断线重连与关闭的生命周期及容错路径。

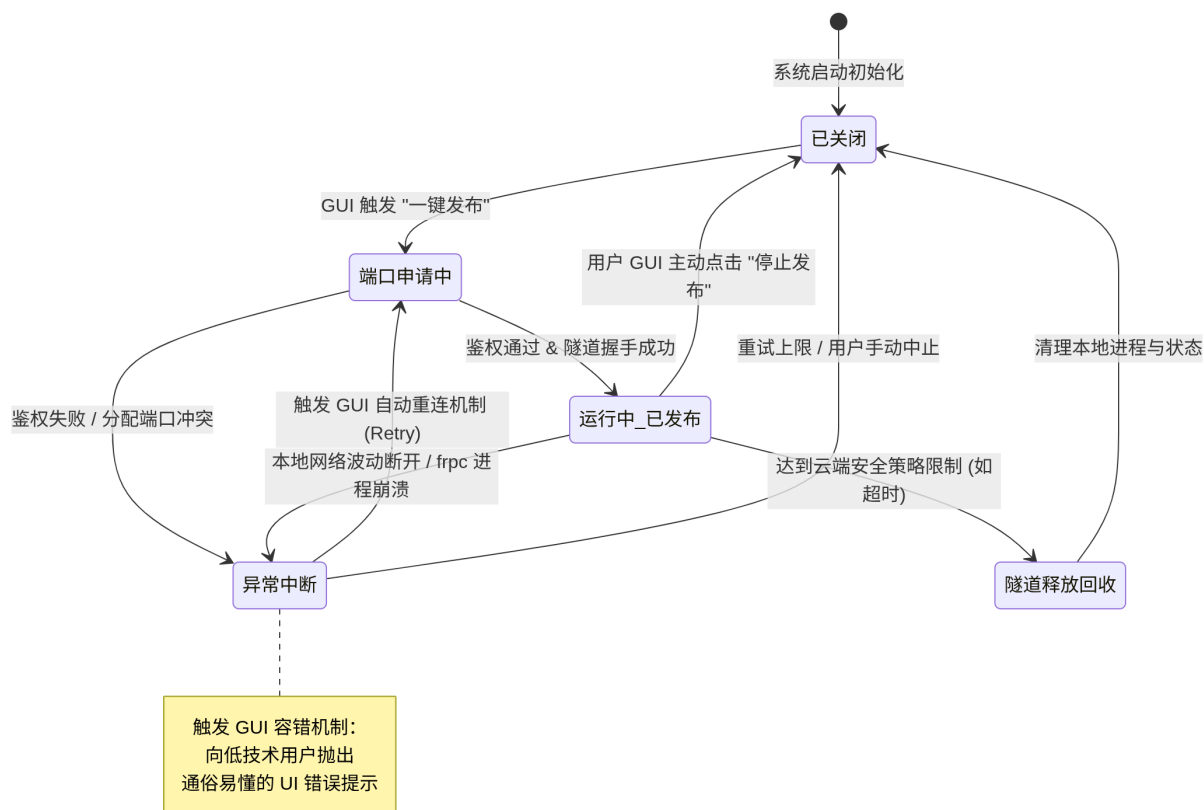


图 3.5.2 发布隧道（TunnelConfig）对象的状态图

3.6 数据设计

WikiBridge 的数据分为**本地数据**与**服务器控制面数据**两部分。系统**明确避免**把用户知识内容上传到服务器持久化保存：用户原始资料、Wiki 页面、本地索引与问答上下文均保留在本地，服务器只保存账号、Token、发布记录与隧道状态，以此满足非功能性需求中的隐私与安全约束。

3.6.1 本地数据结构

用户端在用户选定的工作目录下初始化并维护以下本地数据，见表 3.6.1。

表 3.6.1 用户端本地数据结构

路径 / 文件	数据内容	说明
/raw	原始资料	用户放入的 Word、PDF、网页摘录、笔记、Markdown、TXT 等。
/wiki	Markdown Wiki 页面	编译后的结构化页面，含标题、正文、标签与 [[wiki-links]]。
/index	本地索引	供搜索与 Chatbox 使用的全文 / 向量 / 摘要索引。
config.json	本地配置	素材目录、Wiki 目录、本地端口、模型配置、发布偏好。
material.db	素材状态	文件路径、哈希、更新时间、处理状态、失败原因。
publish.json	发布状态缓存	当前公网 URL、隧道状态、最近连接时间与可回收标识。

3.6.2 服务器控制面数据表

服务器侧的核心数据库表(用户与隧道)及其关联见图 3.6.1:T_User 保存用户账号信息,T_Tunnel 保存每次发布申请到的隧道与公网入口信息，二者通过 user_id 关联。

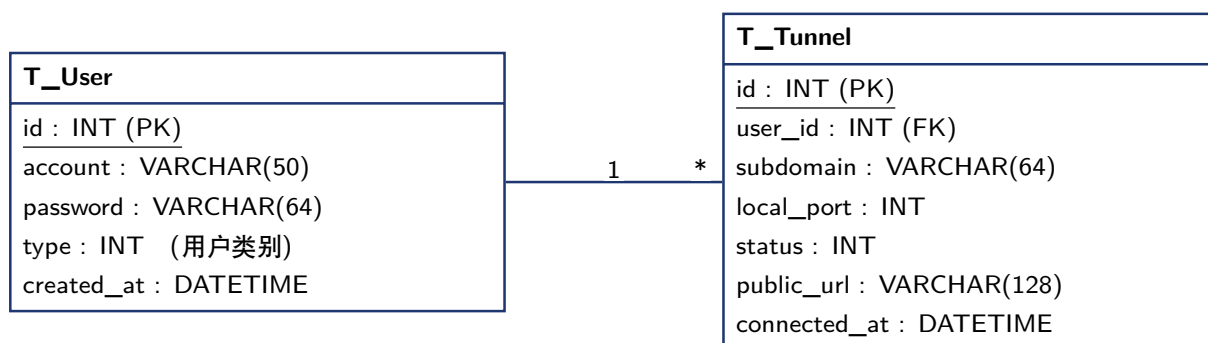


图 3.6.1 控制面数据库表 T_User 与 T_Tunnel

为支撑认证、发布、状态监控与防滥用，服务器侧完整的控制面数据表设计见表 3.6.2，所有表均**不含**任何用户知识内容。

表 3.6.2 服务器控制面数据表设计

数据表	字段说明
T_User	id、account、password_hash、display_name、status、created_at、last_login_at。保存平台账号，不保存知识内容。
T_AccessToken	id、user_id、token_hash、expire_at、revoked_at。用于 Token 校验与吊销。
T_PublishRecord	id、user_id、subdomain、public_port、local_port、status、created_at、closed_at。记录每次发布入口。
T_TunnelStatus	id、publish_id、online、bandwidth_in、bandwidth_out、connection_count、last_heartbeat。管理 frp 隧道运行状态。
T_RateLimitPolicy	id、user_id、max_bandwidth、max_connections、enabled。基础限速与防滥用策略。

3.6.3 永久数据的操作

为支持对上述数据库表的操作,设计模型提供两个仓储类 UserRepository 与 TunnelRepository, 其接口如下:

- boolean insertUser(User u) / boolean deleteUser(User u) / boolean updateUser(User u)
- User getUserByAccount(String account) / boolean verifyUserValidity(String acc, String pwd)
- boolean insertTunnel(TunnelConfig t) / boolean updateStatus(String id, int status)
- List<TunnelConfig> listByUser(int userId) / void closeTunnel(String id)
- void openDatabase() / void closeDatabase()

3.6.4 数据安全设计

为保护用户隐私与账号安全,数据设计遵循以下约束:

- 本地 config.json 中的 Token 应**加密保存**或使用操作系统的安全存储能力,不以明文长期驻留。
- 日志中**不得输出**完整 Token、用户密码、原始资料全文以及模型请求的完整上下文。
- 服务器侧密码仅保存**哈希值**(password_hash),Token 仅保存哈希或可验证摘要(token_hash)。
- 发布入口支持用户主动关闭,管理员也可在滥用或异常流量场景下回收入口 (revoked_at / closed_at)。

3.7 部署设计

WikiBridge 采用分布式部署（见图 3.7.1）。用户端软件（GUI、知识编译器、本地 Web Server、魔改 frpc）部署在知识拥有者的个人主机上；服务器端（管理 API、frps、控制面数据库）部署在公网云主机上；知识消费端为任意带浏览器的设备。用户端 frpc 与服务器 frps 之间通过 frp 隧道连接，消费端浏览器经服务器公网入口被转发回用户端本地服务，从而在**不暴露用户主机**的前提下完成访问。

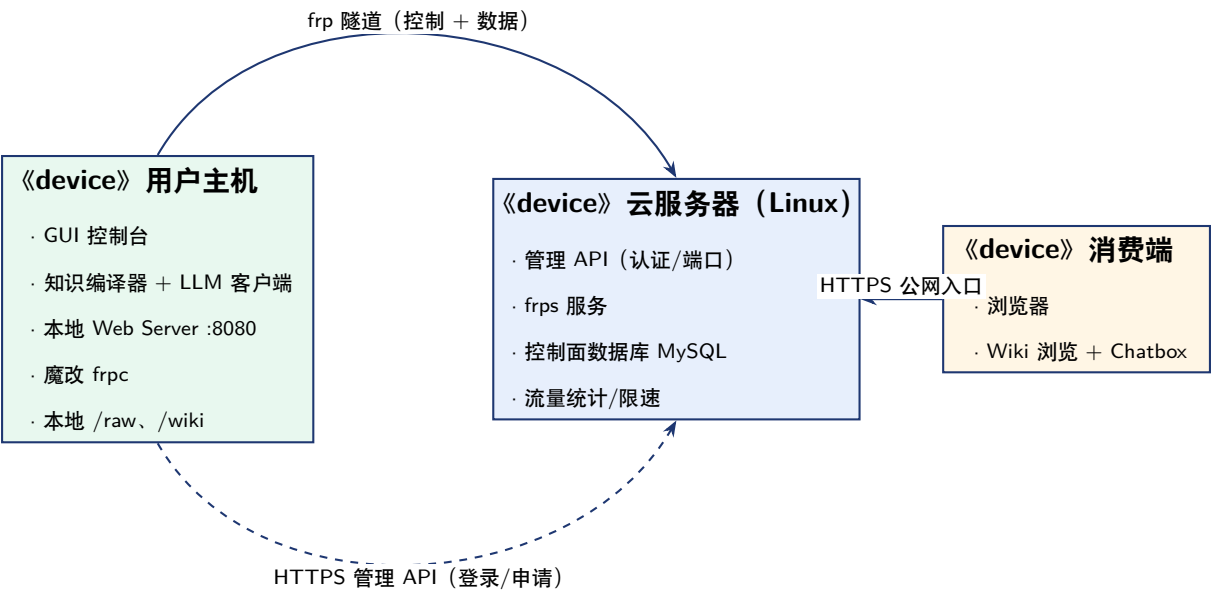


图 3.7.1 WikiBridge 系统的部署图

3.8 异常处理与可靠性设计

为满足非功能性需求中的可靠性，系统针对编译、网络与发布链路上的典型异常给出检测与处理策略，见表 3.8.1。设计原则是**失败隔离、状态可见、可恢复**：单个素材或单次调用失败不应阻塞整体流程，所有异常都应在界面给出可理解的提示而非仅输出底层日志。

表 3.8.1 异常处理与可靠性设计

异常场景	检测方式	处理策略
素材解析失败	解析器返回错误或文本为空	标记素材失败，不阻塞其他素材编译，允许查看原因并重试。
大模型调用失败	超时、限流、返回格式不合法	记录日志，按任务粒度重试，必要时保留半成品并提示用户。
Wiki 链接异常	Lint 扫描到死链接或孤立页面	给出修复建议，涉及删除或合并时等待用户确认。
本地端口占用	本地 Web Server 启动失败	自动建议备用端口，并同步更新 frpc 配置。

异常场景	检测方式	处理策略
Token 过期	服务器返回未授权	提示重新登录，暂停发布操作但不影响本地浏览。
隧道断开	frpc 心跳或 frps 状态异常	自动重连并在界面展示状态；多次失败后提示检查网络。
公网访问滥用	流量与连接数超过阈值	服务器限速、临时关闭入口或要求用户重新确认发布。

3.9 测试设计

课程设计原型阶段建议按以下端到端场景进行验收：

1. **素材箱测试**：将多种格式资料加入 `/raw`，确认系统能识别新增素材并展示状态。
2. **编译测试**：触发 Compile 后生成多个 Markdown 页面，标题清晰且存在合理双向链接。
3. **图谱检查测试**：构造死链接、孤立页面和重复主题，确认 Lint 能识别并给出建议。
4. **本地浏览测试**：启动本地 Web Server 后，浏览器可打开 Wiki 首页、页面详情与搜索结果。
5. **发布测试**：登录后申请公网 URL，外部浏览器可通过该 URL 访问本地 Wiki。
6. **问答测试**：在 Chatbox 中提问，系统返回回答与引用来源，引用可跳转到对应页面。
7. **隐私测试**：检查服务器数据库与日志，确认未保存用户原始资料、Wiki 正文或问答上下文。
8. **异常测试**：模拟端口占用、网络断开、Token 过期与模型调用失败，确认界面提示清晰且可恢复。

结论

WikiBridge 的设计目标是验证“**LLM-Wiki 本地编译 + frp 无感穿透**”的端到端可行性。系统通过 **控制面与数据面分离**、**知识编译与 frp 管理解耦**两条主线，在三端架构下实现了“本地编译 + 无感穿透”的完整闭环：用户端承担知识内容处理与本地托管，服务器只承担认证、端口分配与隧道调度，消费端以浏览器轻量浏览和问答。用户数据与主要计算过程留在本地，云端只做轻量调度，从而同时兼顾**低门槛**、**隐私性**、**低成本与可维护性**。后续实现应优先打通从本地素材到 Markdown Wiki、从本地 Web Server 到公网 URL 的主链路，再逐步完善 Lint 自动修复、访问控制、限速策略与跨平台安装体验。